Real-Time Crowd Commander

Natural-language command of agent crowds in strategy games, under real-time compute budgets

Real-time strategy games bury their best part, the strategy, under hundreds of manual actions a minute. This agenda asks whether the player can instead command in natural language, like a real commander, while a language model turns the orders into actions at game speed. The hard part is cost: what command takes in latency and memory under a clock that never pauses.

> **How much does natural-language command cost at game speed, and how can that cost be driven down?**

A project proposal grown from Yubo Huang's interest and insight, under the guidance of Dr. Enmao Diao.

June 2026

1.1 The Problem and the Vision

I play real-time strategy games: Age of Empires IV, StarCraft II. The decision-making is the fun part: where to expand, when to attack, how to read the opponent.

But that thinking is buried under frantic manual labour: hunting for unit icons, drag-selecting, clicking hundreds of times a minute. The interface, not the strategy, is where the effort goes.

A strategy player should command the way a real commander does:

- **watch** the battle and **listen** to reports,
- **think**,
- **speak** the orders, pressing a key only now and then.

Subordinates handle the execution. That interface could be the next revolutionary strategy game.

Figure 1: the labour the interface forces. A montage of StarCraft II tasks: move here, gather there, build these. Age of Empires IV plays differently but shares the trait, rich strategy buried under hundreds of manual actions a minute.

Still: PySC2 video (DeepMind)

1.2 The Missing Piece: Speed

The commander interface is not a new dream. It shipped once: Tom Clancy's EndWar (2008) was a fully voice-commanded strategy game. But it ran on a rigid 70-word grammar: players had to memorize exact phrases, and anything outside the script failed.

Language models removed that limit. They understand free-form commands. What they cannot yet do is **act at game pace**: a model that takes seconds to answer is useless against an opponent moving every frame.

So the one missing piece is speed, and it sets the research question:

> **How much does natural-language command cost, in latency and memory, at game speed, and how can that cost be driven down?**

Everything is judged one way: **performance as a function of latency budget and memory budget**, an efficiency frontier rather than a single score. The clock never pauses, so a decision that arrives late simply does not happen.

2 The Gap: Efficiency, Untested Where It Counts

LLMs can already play real-time strategy through a text interface: TextStarCraft II (NeurIPS 2024) beats the built-in AI. But systems like it slow or pause the clock to think, so the real-time question is sidestepped, not answered.

And the methods that would make real time possible are tested in the wrong place. Take **KV-cache eviction** (dropping old context to fit a memory budget): it is scored on perplexity and document retrieval, never inside a live game, where evicting the wrong memory, the opponent's scouted tech switch, loses the match minutes later, with no perplexity number to warn you.

The gap, in one line: a streaming game is the most natural stress test for efficient inference, and nobody has run it. Context grows every second, old information matters unevenly, and the ground-truth metric, win or lose, is external and unforgiving.

The field already feels the pressure: the LLM Game Agents Survey (CSUR 2026) names low-latency control as a core open challenge. The opening is visible to everyone.

3 Preliminaries

The background this work builds on:

Area	What it gives us
Reinforcement learning	The substrate: a game is a Markov decision process, win rate is the reward, recalling a scouted tech switch is credit assignment. Phase 1 needs RL *literacy*, not training.
Efficient LLM inference	KV-cache eviction (H2O, SnapKV, StreamingLLM, OBCache), structured pruning, quantization, distillation: the methods the benchmark scores.
Vision-language-action models	π0: a slow vision-language backbone driving a fast action expert, trained by imitation. The template for the commander/executor split.
The StarCraft II agent stack	PySC2, TextStarCraft II, LLM-PySC2: the environment we inherit rather than rebuild.
Tokenization beyond text	Byte-pair encoding and graph tokenization (ICLR 2026): the basis for a learned game-state tokenizer.

4.1 The Three Jobs

The system needs three capabilities: separate lines of work that eventually run together as one system.

Job	What it does
Decide	An LLM commander reads the game-state stream and issues the orders, under hard latency and memory budgets
Foresee	A compact world model answers "what if" before you commit: if the army pushes now, does the fight win?
Embody	Units carry out the orders with model-generated motion (synthesized for the command, not replayed clips) at crowd scale on one GPU

This proposal is about the first job, **Decide**. Foresee and Embody are the growth surface, built later.

4.2 The Plan: Three Phases

The three jobs are the *parts*. The three phases are the *order*: we build in increasingly complex environments, with one shared **harness** under all of them (the command protocol, the unpausable clock, deadline enforcement, metric logging), so the engineering transfers upward.

Phase	Focus	What happens
Phase 1	Benchmarks	Build the harness and the efficiency-frontier evaluation, on two testbeds: a command arena (warm-up) and StarCraft II with the clock unpaused (the flagship).
Phase 2	Methods	Attack whatever Phase 1 exposes as the bottleneck: game-aware eviction, a learned state tokenizer, distilled commanders.
Phase 3	The real interface	Bring in the human (voice), then humans plural (multiplayer competition); grow the Foresee and Embody jobs.

The environments grow with the phases: **a toy room → StarCraft II → multiplayer → a full game.** The end-state game is the north star, not a deliverable: its role is to fix the two constraints every paper inherits: unpausable clock, hard compute budget.

5.1 The Command Arena

Streamed commands

t = 0.0 s **"Gray, walk south."**

t = 1.2 s **"Yellow, sit down."**

t = 2.1 s **"Blue, run north."**

Difficulty rises along three axes: command rate, compositional addressing ("everyone except the yellow one..."), and commands about remembered state ("the one who was sitting earlier...").

The clock never pauses: a late action simply does not happen.

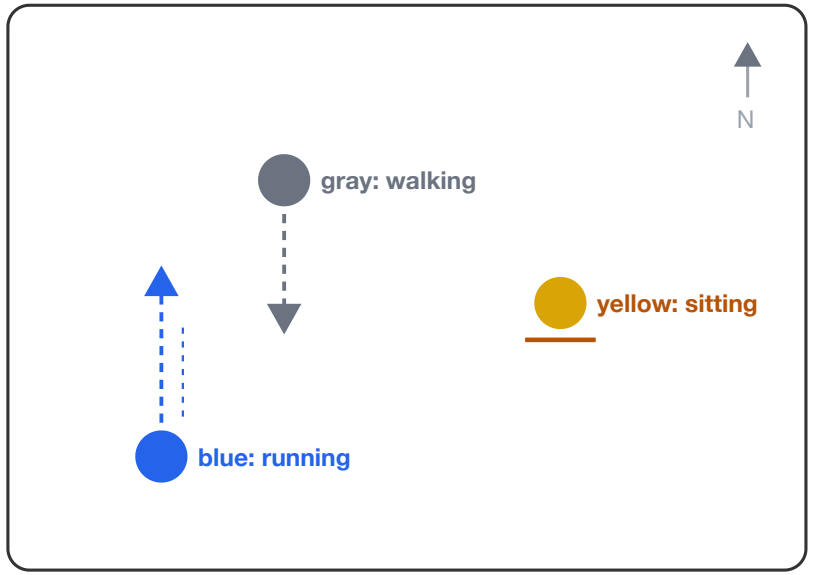

Figure 2: Phase 1's warm-up testbed. Colour-tagged agents in a room, each command colour-keyed to its agent. One command is trivially easy for any modern model, by design: the test is the stream. Difficulty rises with command rate, with compositional orders ("everyone except the yellow one, gather at the door"), and with orders that depend on memory ("the one who was sitting earlier, move west"). It costs weeks, not months, and proves out the harness everything later reuses.

5.2 The StarCraft II Benchmark

The flagship of Phase 1, the **wedge**: a deliberately narrow, fast first paper that splits the agenda open. It asks: can an LLM command at game speed, and which efficiency techniques preserve its judgment?

The setup. Build on the TextStarCraft II stack, but never pause the clock. Every decision carries a wall-clock deadline; the model runs under a fixed memory ceiling. A late decision does not happen.

What varies. The cache policy (full cache vs. methods like StreamingLLM sinks or OBCache (ICML 2026)); model size and sparsity (1B to 70B, pruned, quantized); the architecture; and how game state is encoded.

What we measure. Win rate as a function of latency budget and memory budget. The headline result is a frontier, not one number.

The clever part: strategic-memory probes. Scripted matches winnable only by recalling something seen minutes earlier. They turn an invisible cache mistake into a visible lost game, a report card no perplexity benchmark can give.

6.1 What This Produces

The agenda is research-first: the game is the north star, the papers are the milestones. Each must answer a question its community already cares about, while caring nothing about the game.

Paper	Phase	The question it answers	Who cares, beyond the game
Command-arena benchmark	1	How does grounding degrade as command rate rises?	Real-time agent and interactive-systems researchers
Real-time commander benchmark (the wedge)	1	Which efficiency methods survive a closed-loop game clock?	KV-cache and pruning researchers, whose methods are never scored by win rate
Game-state tokenizer	1 to 2	Does compact tokenization extend to entity and event streams?	The tokenization-beyond-text programme
Competitive-efficiency study	3	Does a fast small commander beat a slow large one?	Inference-efficiency and agents communities
Crowd motion under budget	3	Can language-commanded crowds move in real time on one GPU?	Motion-generation and graphics community

6.2 The Multiplayer Endgame

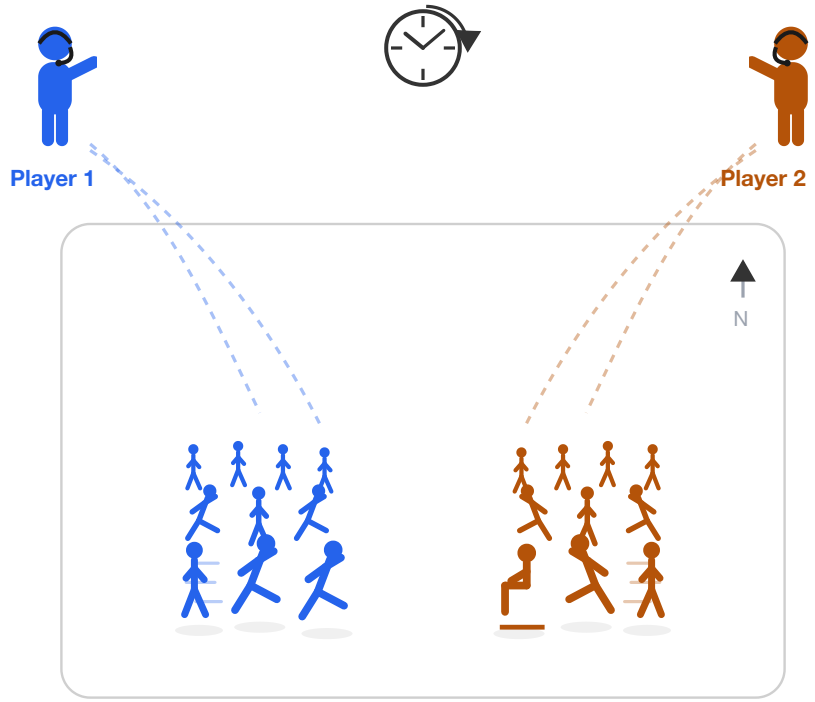

Figure 3: the multiplayer endgame. Several players each command their own army by voice in one shared, unpausable world. Latency stops being a number we measure and becomes what decides the match: does a fast small commander beat a slow large one? Efficiency, settled by Elo.

7 Open Questions

The calls to make together:

- **Scope of the first paper.** The benchmark alone, or the benchmark plus the learned state tokenizer?
- **Compute.** The efficiency-frontier sweep multiplies models, budgets, and matches: what scale is realistic, and on which GPUs?
- **Venue and timing.** A faster ICLR 2027 submission, or a stronger one at NeurIPS 2027?
- **Beyond the papers.** Does the commander interface become a DreamSoul product, or stay a research line?

Full proposal and decisions log: `DreamSoul-AI/game-commander`.